ICT162

Examination - January Semester 2024

Object Oriented Programming

Thursday, 25 April 2024 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **4** questions and comprises **11** printed pages (including cover page).
2. Answer **ALL** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.
5. A4 Sized Cheat Sheet are allowed to be brought into exam venue.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT162 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Analyze the python classes in Figure Q1 to demonstrate your understanding of how class hierarchy and association are used in a typical object-oriented application.

```
1  class ClassA:
2      def __init__(self, x: int):
3         self._varA = x
4
5      def methodA(self, num: int = 5):
6         return self._varA * num
7
8      def __str__(self):
9          return "Variable A: {}".format(self._varA)
10
11 class ClassB:
12     def __init__(self, x: int):
13        self._varB = x
14
15     def methodA(self, num: int = 2):
16        return self._varB * num
17
18     def __str__(self):
19         return "Variable B: {}".format(self._varB)
20
21 class ClassC(ClassA):
22     def __init__(self, x: int, y: int, z: ClassB):
23        super().__init__(x)
24        self._varC1 = y
25        self._varC2 = z
26
27     def methodA(self, num: int = 4):
28        return self._varC1 * num
29
30     def __str__(self):
31         return "{}  Variable C1: {}  Variable C2: {}" \
32                .format(super().__str__(), self._varC1, self._varC2)
33
34 class ClassD(ClassA):
35     def __init__(self, x: int, y: int):
36        super().__init__(x)
37        self._varD = y
38
39     def methodA(self):
40        return super().methodA(self._varD)
41
42     def __str__(self):
43         return "Variable D: {}".format(self._varD)
44
45 def main():
46     objA = ClassA(7)
47     objB = ClassB(6)
48     objC = ClassC(5, 4, objB)
49     objD = ClassD(3, 2)
50     classList = [objA, objB, objC, objD]
51     for obj in classList:
52         print( obj.methodA() )
53
54     print("Line 54:", objC)
55
56 main()
```

Figure Q1

Question 1a

Provide an example for each of the terms that follows. Indicate the line number(s) and specify which term is demonstrated by the code on the line number(s):

(i) Reuse by inheritance

(ii) Reuse by object composition

(iii) Method overriding by replacement

(iv) Method overriding by refinement

(v) Polymorphism

(vi) Overloading of method

(18 marks)

Question 1b

State and describe the output produced by the program in Figure Q1.

(7 marks)

Question 2

CCAs (co-curricular activities) are a key component of students' holistic education. Through CCAs, students discover their interests and talents. CCAs also provide platforms for students to develop their character, learn values, social emotional competencies, and other relevant skills to prepare them for future challenges.

In SUSS, there are 2 main groups of CCA: sports and interest. The class diagram in Figure Q2 shows specifications of CCA.

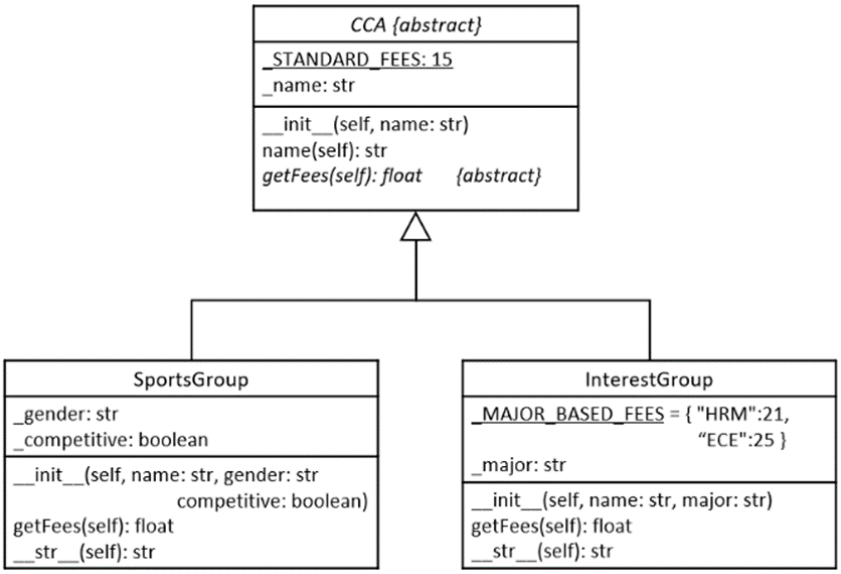


Figure Q2

Question 2a

Implement the abstract class – CCA.

- _STANDARD_FEES: class variable that defines the standard monthly fees that a student has to pay for participating in the CCA.
- It has ONE instance variables:
  - _name (str): the name of the CCA. E.g., “Ultima Frisbee”, “Tennis”, “Dance(X)”, “Artificial Intelligence”
- Constructor initializes the instance variables: _name.
- Getter method for name. Use the property decorator.
- One abstract method, getFees() which currently has no implementation.

(7 marks)

Question 2b

Write and construct Python class definitions for SportsGroup.

The SportsGroup class is a subclass of CCA, and it has:

- Apart from name, constructor also initialises these TWO additional instance variables:
  - _gender (str): specifying if this SportsGroup object is meant for “Male”, “Female” or “Coed”.
  - _competitive (boolean): indicates if this SportsGroup is competitive or recreational.
- The method getFees() returns 0 if the SportsGroup is competitive. Otherwise, the method returns the _STANDARD_FEES.
- The __str__ method returns a string representation of a SportsGroup object, which should include the name, gender, and the monthly fees. These are examples in the suggested format:

```
cca1 = SportsGroup("Ultimate Frisbee", "Coed", False)
cca2 = SportsGroup("Tennis", "Male", True)
print(cca1)
print(cca2)
```

Output:

```
Ultimate Frisbee (Coed), monthly: $15.00
Tennis (Male), monthly: $0.00
```

(8 marks)

Question 2c

Write and construct Python class definitions for InterestGroup.

The InterestGroup class is a subclass of CCA, and it has:

- Apart from name, constructor also initialise ONE additional instance variable:
  - _major (str): specifying which major this InterestGroup focuses on. Default is None.
- ONE additional class variables:
  - _MAJOR_BASED_FEES (dict): defines the pairing of major and its monthly fees, that differs from standard. In Figure Q2, it shows InterestGroup with major as "HRM" will have monthly fees $21, while "ECE" will have monthly fees of $25.
- The method getFees() checks if the major of this InterestGroup is one of those in class variable _MAJOR_BASED_FEES. If it is, the method returns the fees specified in the value of the dictionary. Otherwise, the method returns the _STANDARD_FEES.
- The __str__ method returns a string representation of a SportsGroup object, which should include the name and the monthly fees. These are examples in the suggested format:

```
cca3 = InterestGroup("Playpen for Teachers", "ECE")
cca4 = InterestGroup("Dance(X)")
print(cca3)
print(cca4)
```

Output:

```
Playpen for Teachers, monthly: $25.00
Dance(X), monthly: $15.00
```

(10 marks)

Question 3

Study the class diagram in Figure Q3, which showcase a simplified relationship between Student and CCA. In SUSS, student can only enrol up to 3 CCAs, or allow to pay a max CCA fees of $45.

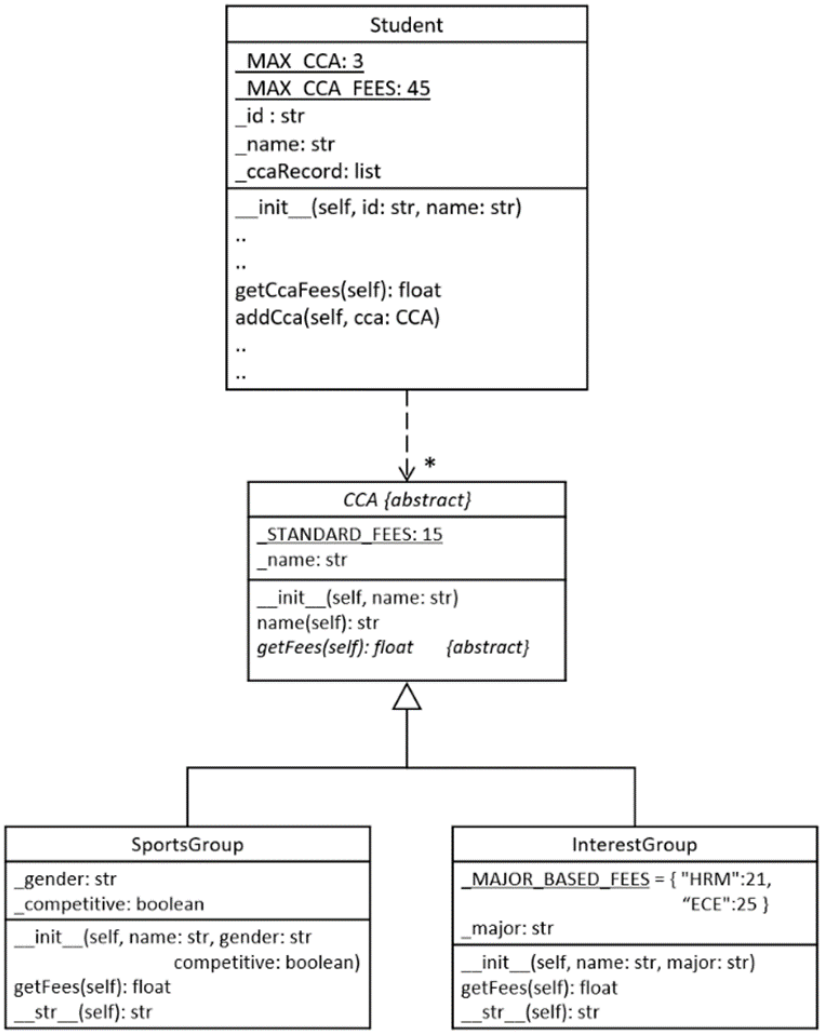


Figure Q3

Assume the Student class in Figure Q3 is partially written.

The Student class has the following attributes:

- TWO class variables:
  - _MAX_CCA: represents the maximum number of CCAs a student can enrolled.
  - _MAX_CCA_FEES: represents the fees ceiling (fees cap) a student should be paying for all enrolled CCAs.

- THREE instance variables:
  - _id is the unique student ID.
  - _name represents the student's name.
  - _ccaRecord is a list containing the enrolled CCA for this student.

The constructor performs the following:

- Initialize _id and _name with the given parameters.
- Set _ccaRecord to an empty list.

Question 3a

By applying the principles of object-oriented programming principles, define the class CcaException, which is a subclass of the Exception class. This class has no additional attribute or method.

(2 marks)

Question 3b

Implement the method getCcaFees() of Student class.

- This method will return the summation of all CCA fees enrolled by the student.

(7 marks)

Question 3c

Implement the method addCca() of Student class.

- Given a new CCA object as parameter, this addCca() method perform the following checks before adding the new CCA into the collection _ccaRecord:
  - If this addition results in _ccaRecord having more CCA count than _MAX_CCA, raise CcaException with message “Cannot enrol more than 3 CCAs”.
  - If this addition results in _ccaRecord having total CCA fees more than _MAX_CCA_FEES, raise CcaException with message “Cannot enrol as total CCA fees will exceed $45”.
  - If there is already an existing CCA with the same name, raise CcaException with message "You already enrolled in this CCA".

(9 marks)

Question 3d

See the following Python codes that create a Student object s and the intention to add CCAs for this student. Complete the Python codes, with the given instructions starting from line 8. Ensure exceptions are effectively managed.

```
1  def main():
2      s = Student("B123", "Bu Ah Ya")
3      cca1 = SportsGroup("Ultimate Frisbee", "Mixed", False)
4      cca2 = SportsGroup("Tennis", "Male", True)
5      cca3 = InterestGroup("Playpen for Teachers", "ECE")
6      cca4 = InterestGroup("Dance(X)")
7
8      # Write codes to add CCAs in this sequence:
9      #     cca1, cca2, cca3, cca4
10     #
11     # Ensure exceptions are properly handled
12
13
14
15
16     # print out the monthly CCA fees for student s
17
18
19 main()
```

(7 marks)

Question 4a

Implement the 2 class methods in InterestGroup class as illustrated below in Figure Q4-1.

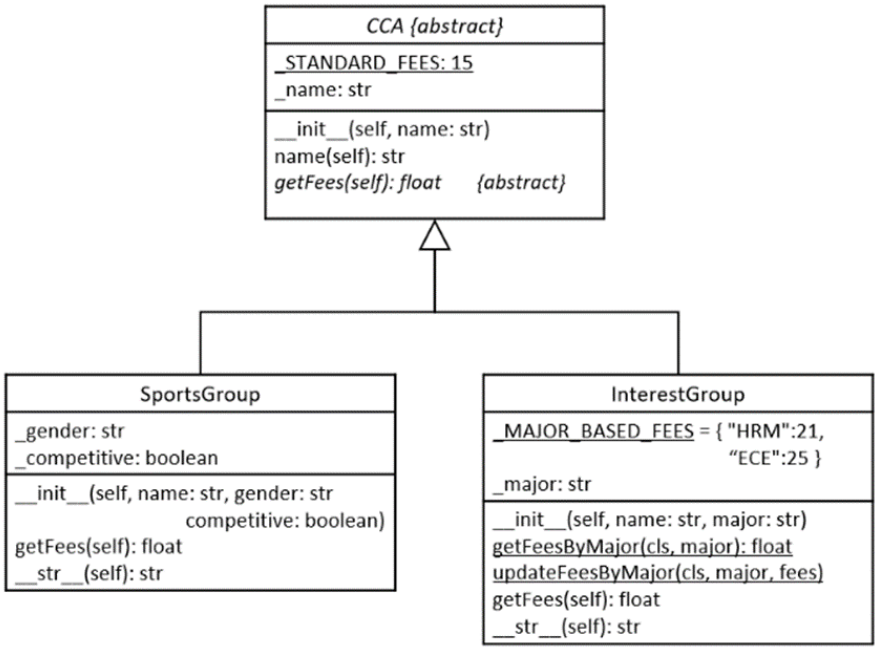


Figure Q4-1

Class method getFeesByMajor() returns the fees based on the given parameter major. If there is no matching major in _MAJOR_BASED_FEES, the method returns None.

Class method updateFeesByMajor() has major and fees as parameters. The method will:

- add the given major and fees into the _MAJOR_BASED_FEES, if the major is not found in the dictionary, or
- if the given major is found in the dictionary, update the fees accordingly.

(8 marks)

Question 4b

A partially developed GUI is showcased below in Figure Q4-2, to allow administrators to check, setup and update the InterestGroup's fees based on the "major".

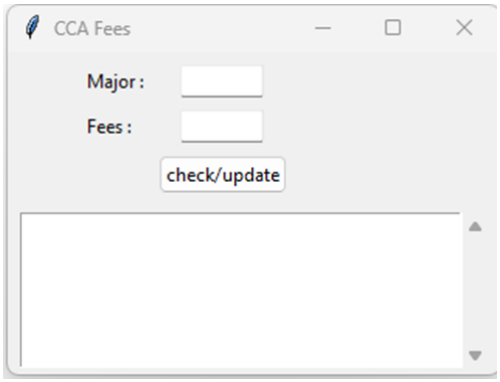


Figure Q4-2

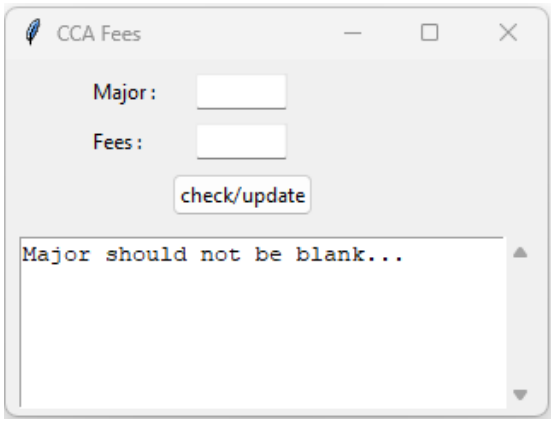
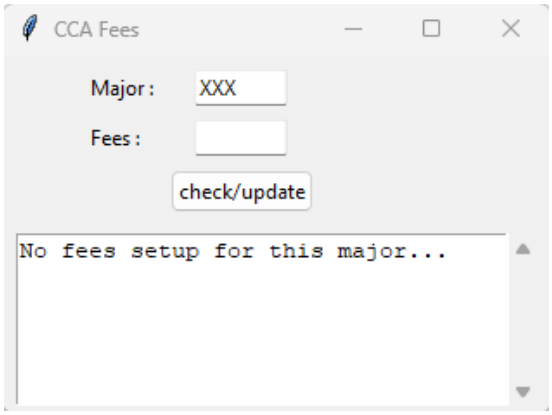
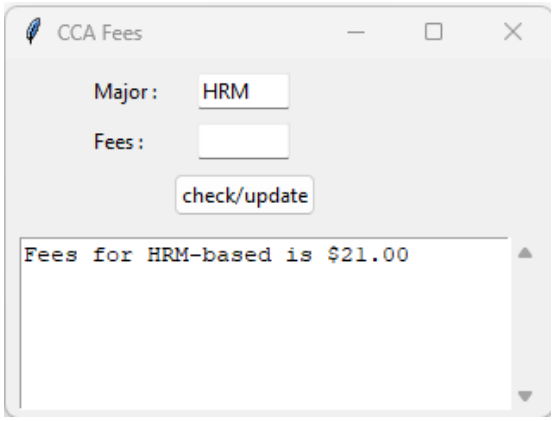
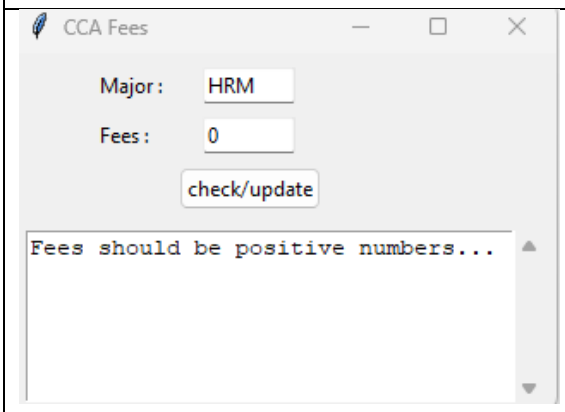
The following is the initial code for the GUI shown in Figure Q4-2a. In line 4, we assumed the class InterestGroup is inside the file CCA_classes.py

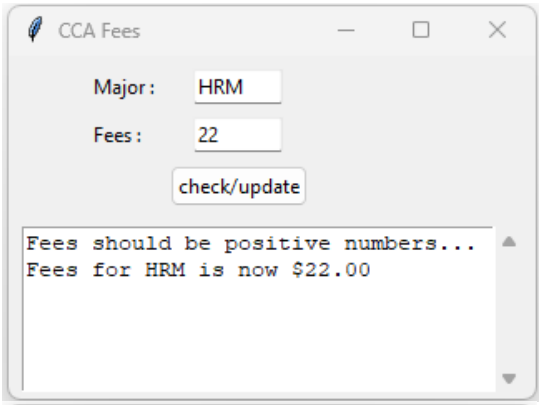
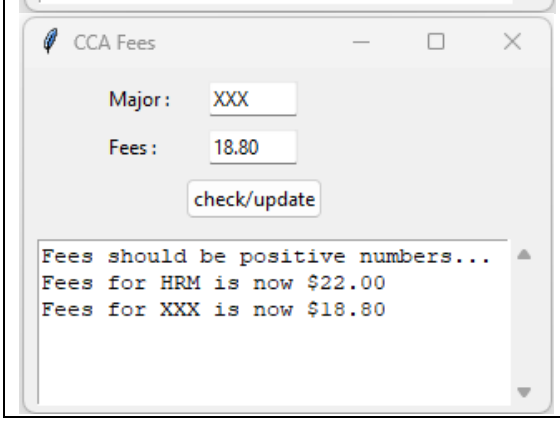
```
1  import tkinter as tk
2  from tkinter import ttk
3  from tkinter import scrolledtext
4  from CCA_classes import InterestGroup
5
6  class CcaFeesGUI:
7
8      def __init__(self):
9          self._win = tk.Tk()
10         self._win.resizable(True, True)
11         self._win.title("CCA Fees")
12         self.create_widgets()
13         self._win.mainloop()
14
15     def create_widgets(self):
16         dataFrame = ttk.Frame(self._win)
17         dataFrame.grid(row=0, column=0, padx=8, pady=4)
18
19         major_lbl = ttk.Label(dataFrame, text="Major : ")
20         major_lbl.grid(row=0, column=0, sticky='W')
21         self._major = tk.StringVar()
22         self._major_Ety = ttk.Entry(dataFrame, width=8,
23                                     textvariable=self._major)
24         self._major_Ety.grid(row=0, column=1, padx=4, pady=4)
25
26         fees_lbl = ttk.Label(dataFrame, text="Fees : ")
27         fees_lbl.grid(row=1, column=0, sticky='W')
28         self._fees = tk.StringVar()
29         self._fees_Ety = ttk.Entry(dataFrame, width=8,
30                                    textvariable=self._fees)
31         self._fees_Ety.grid(row=1, column=1, padx=4, pady=4)
32
33         self._action_btn = ttk.Button(dataFrame, text="check/update",
34                                       command=self.action)
35         self._action_btn.grid(row=2, column=1, padx=4, pady=4)
36
37         outputFrame = ttk.Frame(self._win)
38         outputFrame.grid(row=1, column=0, padx=8, pady=4, columnspan=2)
39         self._scrol_stxt = scrolledtext.ScrolledText(outputFrame, width=35,
40                                                      height=6, wrap=tk.WORD)
41         self._scrol_stxt.grid(row=0, column=0, sticky='WE')
42         self._scrol_stxt.config(state=tk.DISABLED)
43         self._major_Ety.focus()
44
45     def action(self):
46         pass
```

Figure 4-2a

Write statements that will implement the event handler for “check/update” button, in other words, develop and complete the callback function action().

Below are demonstrations of the CcaFeesGUI:

	If the button is clicked when Major is empty, display a message.
 	If major is filled but fees remain empty, this is the scenario that the administrator want to check the existing fees. If the major is not found/setup in InterestGroup, display a message. If the major is found/setup in InterestGroup, display the current fees.
	If both major and fees are filled, this is the scenario that the administrator want to update the fees of the major. Check that fees should be positive, if not, display a message.

 	If the inputs are fine, invoke the appropriate class method to add/update the major and fees into InterestGroup.

(17 marks)

----- END OF PAPER -----